

Introduction to Machine Learning Methods

Principle Component Analysis

Firooz Bashashi Saghezchi
RWTH Aachen University

Outline

- Unsupervised learning
 - Principal Component Analysis (PCA)
 - Anomaly Detection

Reference: Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. (Chapter 12.1)

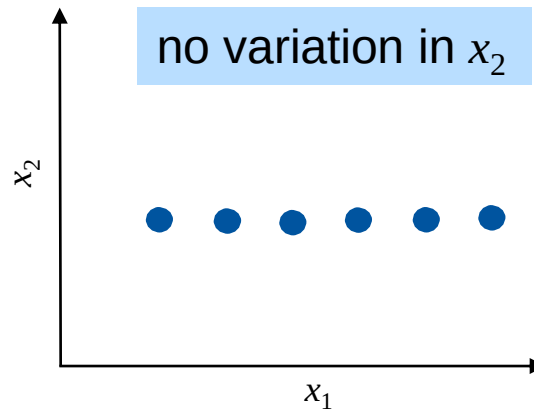
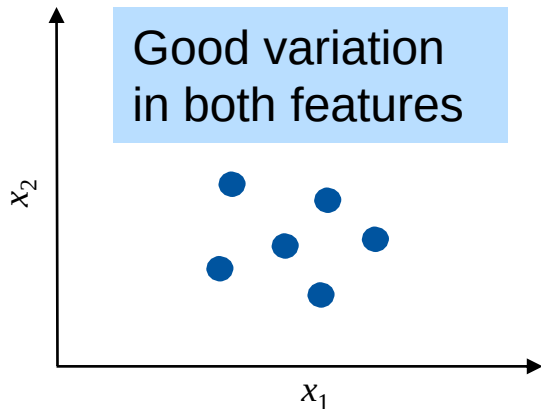
Pattern Recognition and Machine Learning



Introduction to PCA

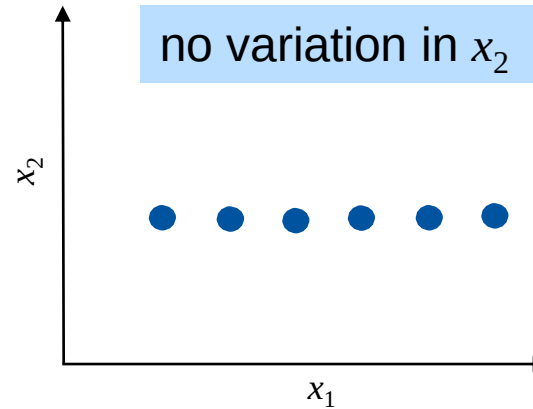
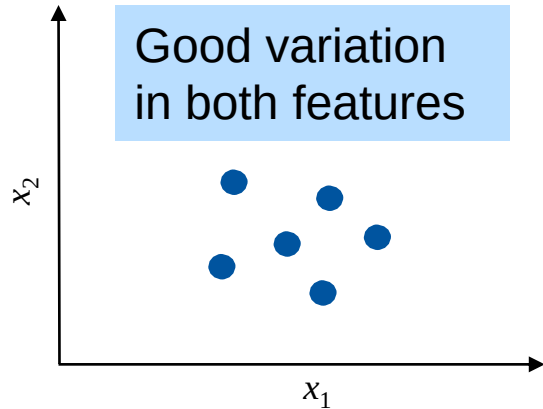
Principal Component Analysis (PCA) – Motivation

- **Problem:** High-dimensional data can contain redundancy
- **Goal:** Reduce dimensions (features) while preserving as much information in the training data as possible
- By “information”, we mean **variance** in the input features $(x_j^{(i)}, j = 1, \dots, n)$



PCA – Motivation (Continued)

- For example, a feature that is constant across all classes is not helpful for a supervised learning algorithm to classify/cluster the data
- Hence, we are interested in features that vary as much as possible across different data classes/clusters



Introduction to PCA

- PCA is an **unsupervised** ML algorithm since it doesn't rely on ground-truth output labels ($y^{(i)}$'s), rather it works only with the input features ($x_j^{(i)}$'s)
- PCA constructs new features ($w_j^{(i)}$'s) which are **linear combinations** of the original features ($x_j^{(i)}$'s)
- Note that though original features may have some meanings (e.g., packet length, packet inter-arrival time), the new features constructed by PCA have **no meaning**

Application of PCA

- **Data Visualization:** reducing high-dimensional data to 2 to 3 features that can be illustrated in 2D or 3D space to visualize the data before training an ML algorithm
- **Dimensionality Reduction:** Eliminating duplicate or less informative features in the training data before training an ML model
- The training data may contain **duplicate features**, for example:
 - Using both kg and pound for describing weight
 - Or, using both km/h and miles/h for describing user speed

Applications of PCA (Continued)

- PCA can automatically find and **eliminate redundant features**
- Hence, applying PCA before training an ML model can **reduce the risk of overfitting**
- **Data Compression:** For example, for image recognition application, a 100x100-pixel gray-scale image contains 10^4 features

Applications of PCA (Continued)

- Using PCA, we can ensure that we construct **less but informative features** for building an ML model
- We may also use PCA to **compress the data** to save on the **storage** space
- However, compression with PCA is **lossy**, and we cannot recover the original data from the compressed data



PCA Algorithm

PCA Algorithm

- Finds new direction(s) that capture **maximum variance** in the data
- Each direction is an **eigenvector** of the **covariance matrix** of the data $\Sigma \in \mathbb{R}^{n \times n}$, a symmetric and semi-definite matrix defines as follows:

$$\Sigma = \frac{1}{m} \sum_{i=1}^m \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)}$$

$$= \frac{1}{m} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$$

$$\tilde{\mathbf{X}} = \begin{bmatrix} \tilde{\mathbf{x}}^{(1)} \\ \vdots \\ \tilde{\mathbf{x}}^{(m)} \end{bmatrix} = \begin{bmatrix} \mathbf{x}^{(1)} - \boldsymbol{\mu} \\ \vdots \\ \mathbf{x}^{(m)} - \boldsymbol{\mu} \end{bmatrix}$$

$$\boldsymbol{\mu} = [\mu_1 \quad \dots \quad \mu_n]$$

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$

Data Covariance Matrix

- The ij^{th} component of the covariance matrix is the covariance between the i^{th} and j^{th} feature in the data, i.e.

$$\sigma_{ij} = \frac{1}{m} \sum_{i'=1}^m \tilde{x}_i^{(i')} \tilde{x}_j^{(i')}$$

Principle Components (Eigenvectors)

- Remember that the eigen vectors of a given matrix $\Sigma \in \mathbb{R}^{n \times n}$ are the vectors $\mathbf{u} \in \mathbb{R}^n$ in the domain of Σ that maintain their direction during the mapping, i.e.

$$\Sigma \mathbf{u} = \lambda \mathbf{u},$$

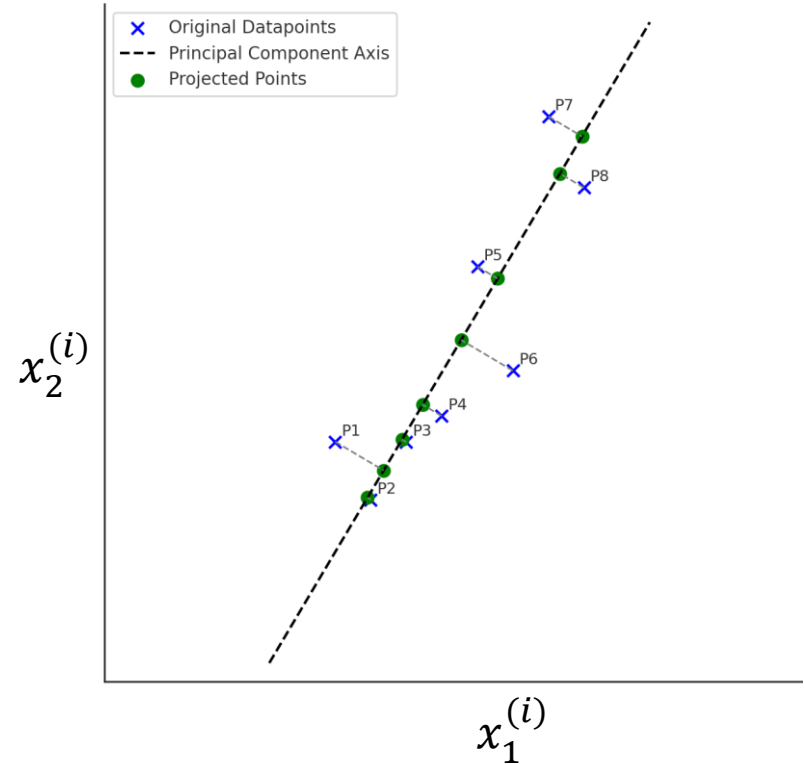
- λ is a scalar, which is called an eigenvalue
- The eigenvector corresponding to the largest eigenvalue is called the **principal eigenvector**

PCA Mapping

- PCA projects data from the original n -dimensional space ($\mathbb{R}^n$) onto a lower-dimensional subspace ($\mathbb{R}^k$) spanned by the top k eigenvectors (i.e. the eigenvectors corresponding to k largest eigenvalues)
- Usually k is much smaller than n ($k \ll n$)

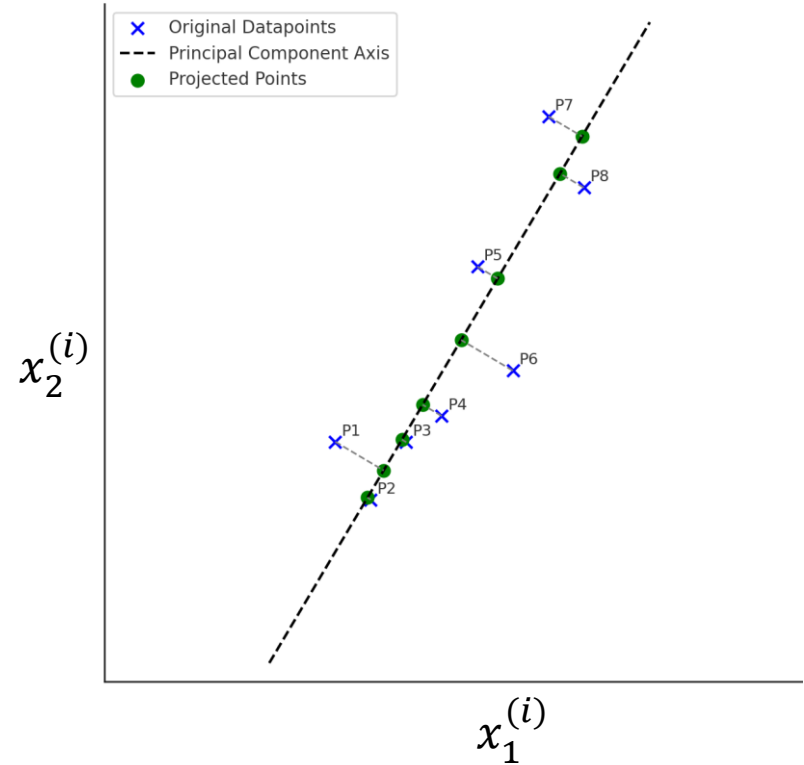
PCA Illustration

- Consider the 2D data illustrated in the figure
- It is easy to see that the two features are highly correlated
- The data varies mostly in the direction of the principal eigen vector (dashed line)



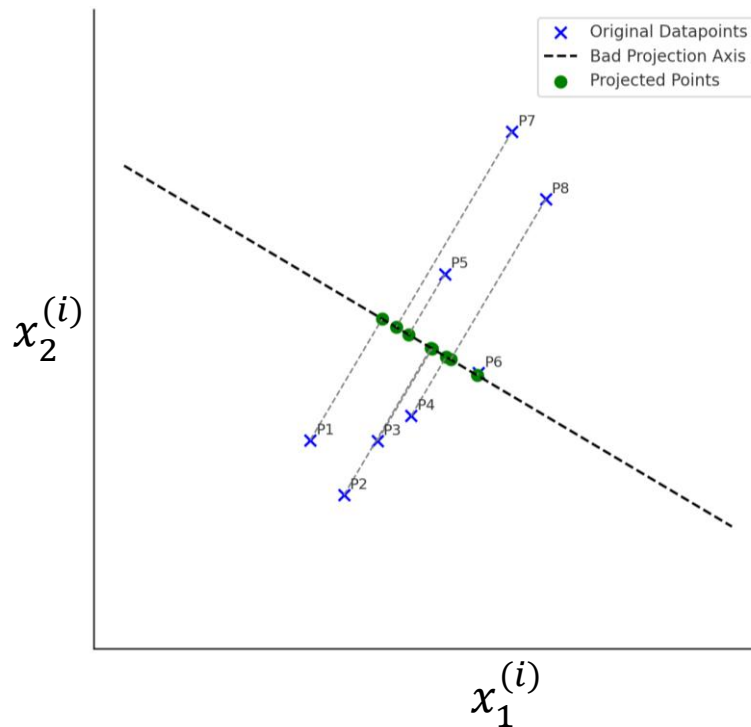
PCA Illustration

- Consider the 2D data illustrated in the figure
- It is easy to see that the two features are highly correlated
- The data varies mostly in the direction of the principal eigen vector (dashed line)
- The projection of data over this direction reduces the data from 2D (blue crosses) to 1D (green dots), while still preserving the maximum variance in the data



Bad Projection Direction

- On the other hand, projection of the data in the direction shown on the figure is a bad choice and should be avoided
- This is because of that fact that such projection leads to a considerable amount of loss in data variance



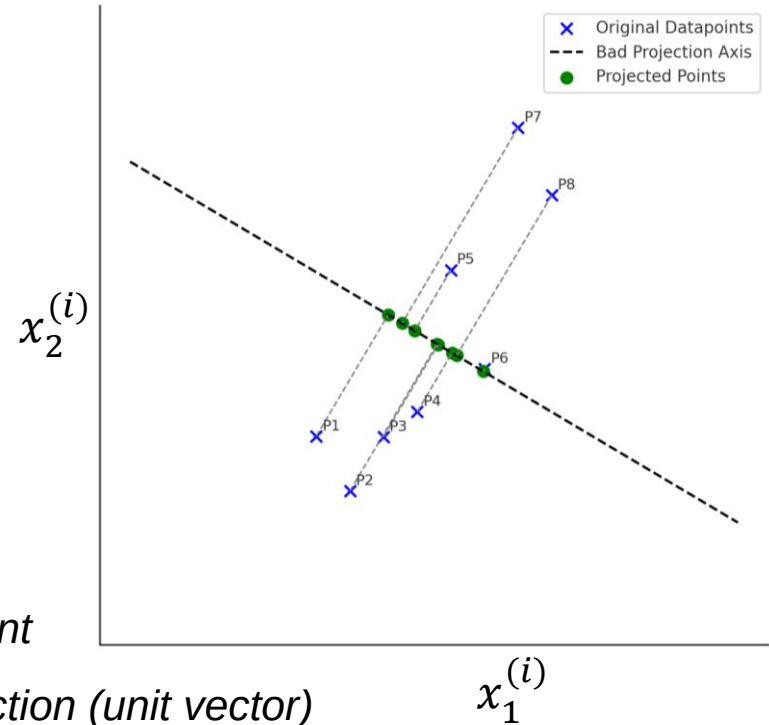
Bad Projection Direction

- On the other hand, projection of the data in the direction shown on the figure is a bad choice and should be avoided
- This is because of that fact that such projection leads to a considerable amount of loss in data variance
- For a given unit vector $\mathbf{u}_j \in \mathbb{R}^n$ ($j = 1, \dots, k$), the projection of a datapoint $\mathbf{x}^{(i)}$ onto that direction is simply given by their dot product:

Mapped point $w_j^{(i)} = \mathbf{x}^{(i)} \cdot \mathbf{u}_j$

Original datapoint (points to $\mathbf{x}^{(i)}$)

Projection direction (unit vector) (points to $\mathbf{u}_j$)





Mathematical formulation of PCA

PCA Optimization Problem

- In order to find the unit vector $\mathbf{u} \in \mathbb{R}^n$ indicating the direction in which the data varies the most, we should solve the following optimization problem

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2$$

Subject to: $\|\mathbf{u}\| = 1$

Variance of the projected data

PCA Optimization Problem

- Since we have subtracted the mean, $\tilde{\mathbf{x}}^{(i)}$ has zero mean
- The projected data also has zero mean since it is a linear transformation of the original data
- Hence, the argument of the objective function above is just the variance of the projected data onto the direction of $\mathbf{u}$

PCA Optimization Problem

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2$$

Subject to: $\|\mathbf{u}\| = 1$

PCA Optimization Problem

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2$$

Subject to: $\|\mathbf{u}\| = 1$

- Note that in this optimization problem we have to constraint $\mathbf{u}$ to be a unit vector, otherwise the objective function can grow arbitrarily large by increasing the length of $\mathbf{u}$

PCA Optimization Problem

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2$$

Subject to: $\|\mathbf{u}\| = 1$

- Note that in this optimization problem we have to constraint $\mathbf{u}$ to be a unit vector, otherwise the objective function can grow arbitrarily large by increasing the length of $\mathbf{u}$
- In order to solve this problem, we use the fact that the norm squared of a given vector $\mathbf{u}$ is equal to the dot product of the vector to itself

$$\|\mathbf{u}\|^2 = \mathbf{u} \cdot \mathbf{u} = \mathbf{u} \mathbf{u}^T$$

PCA Optimization Problem

- Using this identity, we can rewrite the argument of the summation as follows:

$$\|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2 = (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)(\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T) = (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)^T (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)$$

PCA Optimization Problem

- Using this identity, we can rewrite the argument of the summation as follows:

$$\|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2 = (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)(\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T) = (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)^T (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)$$

- Using the identity $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$, we can rewrite the above equation as follows:

$$\|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2 = \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T$$

PCA Optimization Problem

- Using this identity, we can rewrite the argument of the summation as follows:

$$\|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2 = (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)(\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T) = (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)^T (\tilde{\mathbf{x}}^{(i)} \mathbf{u}^T)$$

- Using the identity $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$, we can rewrite the above equation as follows:

$$\|\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}\|^2 = \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T$$

- Now, using this result, we can rewrite our original optimization problem as follows:

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T$$

objective function

Subject to: $\mathbf{u} \mathbf{u}^T = 1$ *constraint*

Solving PCA Optimization Problem

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T$$

$$\text{Subject to: } \mathbf{u} \mathbf{u}^T = 1$$

Solving PCA Optimization Problem

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T$$

$$\text{Subject to: } \mathbf{u} \mathbf{u}^T = 1$$

- In order to solve this equation, we construct the Lagrangian function

$$L(\mathbf{u}, \lambda) = \frac{1}{m} \sum_{i=1}^m \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T + \lambda(1 - \mathbf{u} \mathbf{u}^T)$$

Standard form of optimization problem:

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_i(x) = 0, \quad i = 1, \dots, p, \end{array}$$

Lagrangian:

$$L(x, \lambda, \nu) = f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^p \nu_i h_i(x)$$

Solving PCA Optimization Problem

$$\max_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T$$

$$\text{Subject to: } \mathbf{u} \mathbf{u}^T = 1$$

- In order to solve this equation, we construct the Lagrangian function

$$L(\mathbf{u}, \lambda) = \frac{1}{m} \sum_{i=1}^m \mathbf{u} \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u}^T + \lambda(1 - \mathbf{u} \mathbf{u}^T)$$

- We then take its derivative with respect to $\mathbf{u}$ and set it equal to zero:

$$\nabla_{\mathbf{u}} L = \frac{\partial L}{\partial \mathbf{u}} = \frac{2}{m} \sum_{i=1}^m \tilde{\mathbf{x}}^{(i)T} \tilde{\mathbf{x}}^{(i)} \mathbf{u} - 2\lambda \mathbf{u} = 2(\Sigma \mathbf{u} - \lambda \mathbf{u}) = \mathbf{0}$$

Covariance matrix of data

PCA Optimization Solution

- Solving this equation, yields

$$\mathbf{\Sigma} \mathbf{u} = \lambda \mathbf{u}$$

- Which means that $\mathbf{u}$ is an eigenvector of the covariance matrix $\mathbf{\Sigma}$

PCA Optimization Solution

- Solving this equation, yields

$$\Sigma \mathbf{u} = \lambda \mathbf{u}$$

- Which means that $\mathbf{u}$ is an eigenvector of the covariance matrix Σ
- Furthermore, maximizing the objective function imposes that the Lagrange multiplier λ must be the largest eigenvalue of Σ
- To choose k most informative features, we need to find the top k eigenvectors of Σ and map our data in the subspace spanned by them



Summarizing PCA algorithm

Centering Data Around its Mean

Step 1: Centering the data around its mean (for each feature, subtract the mean) – we can do this since the mean (DC) value doesn't contain any information

$$\boldsymbol{\mu} = [\mu_1 \quad \cdots \quad \mu_n]$$

$$\tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{x}^{(1)} - \boldsymbol{\mu} \\ \vdots \\ \mathbf{x}^{(m)} - \boldsymbol{\mu} \end{bmatrix}$$

Calculating Data Covariance Matrix

Step 1: Centering the data around its mean (for each feature, subtract the mean) – we can do this since the mean (DC) value doesn't contain any information

$$\boldsymbol{\mu} = [\mu_1 \quad \cdots \quad \mu_n]$$

$$\tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{x}^{(1)} - \boldsymbol{\mu} \\ \vdots \\ \mathbf{x}^{(m)} - \boldsymbol{\mu} \end{bmatrix}$$

Step 2: Calculate the data covariance matrix, which is an $n \times n$ symmetric matrix

$$\boldsymbol{\Sigma} = \frac{1}{m} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$$

Finding Eigenvalues

Step 3: Find the n eigenvalues of Σ and sort them in descending order $\lambda_1, \dots, \lambda_n$

Finding the Top k Eigenvectors

Step 3: Find the n eigenvalues of Σ and sort them in descending order $\lambda_1, \dots, \lambda_n$

$$|\Sigma - \lambda \mathbf{I}| = 0$$

Step 4: Chose the first k (largest) eigenvalues and find their corresponding eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ by solving the following equation

$$\Sigma \mathbf{u} = \lambda \mathbf{u}$$

Normalizing the Top k Eigenvectors

Step 3: Find the n eigenvalues of Σ and sort them in descending order $\lambda_1, \dots, \lambda_n$

$$|\Sigma - \lambda \mathbf{I}| = 0$$

Step 4: Chose the first k (largest) eigenvalues and find their corresponding eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ by solving the following equation

$$\Sigma \mathbf{u} = \lambda \mathbf{u}$$

Step 5: Normalize these top k eigenvectors, so they all have unit length, that is replace $\mathbf{u}_j$'s with $\frac{\mathbf{u}_j}{|\mathbf{u}_j|}$

Note: the unit vectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ form an orthonormal basis spanning a k -dimensional subspace of the original n -dimensional feature space (i.e. the PCA subspace)

Projecting Data onto Normalized Top k Eigenvectors

Step 6: Project the original training examples to the subspace spanned by these k eigenvectors

$$\mathbf{w}^{(i)} = [w_1^{(i)} \quad \dots \quad w_k^{(i)}] = [\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}_1 \quad \dots \quad \tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}_k]$$

Note: The basis vectors $\mathbf{u}_j$'s have the same dimensionality as the original datapoints $\tilde{\mathbf{x}}^{(i)}$'s (i.e. $\mathbf{u}_j \in \mathbb{R}^n$). However, we select k of them $\mathbf{u}_1, \dots, \mathbf{u}_k$, which span the PCA subspace. Any vector $\mathbf{w}^{(i)}$ in this subspace can be expressed as a linear combination of these basis vectors:

$$\mathbf{w}^{(i)} = \sum_{j=1}^k (\tilde{\mathbf{x}}^{(i)} \cdot \mathbf{u}_j) \mathbf{u}_j$$

Note: Each basis vector lies in $\mathbb{R}^n$, but a set of $k < n$ such vectors spans only a k -dimensional subspace. For example, in $\mathbb{R}^3$, the vectors $(1,0,0)$ and $(0,1,0)$ span the 2D xy -plane



Numerical Example

Numerical Example

Dataset:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)} \\ \mathbf{x}^{(2)} \\ \mathbf{x}^{(3)} \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 9 \end{bmatrix}$$

Calculate the mean vector:

$$\boldsymbol{\mu} = [\mu_1 \quad \mu_2] = [1 \quad 3]$$

Subtract the mean from every data point because it doesn't contain any information:

$$\tilde{\mathbf{X}} = \begin{bmatrix} \tilde{\mathbf{x}}^{(1)} \\ \tilde{\mathbf{x}}^{(2)} \\ \tilde{\mathbf{x}}^{(3)} \end{bmatrix} = \begin{bmatrix} \mathbf{x}^{(1)} - \boldsymbol{\mu} \\ \mathbf{x}^{(2)} - \boldsymbol{\mu} \\ \mathbf{x}^{(3)} - \boldsymbol{\mu} \end{bmatrix} = \begin{bmatrix} -2 & -4 \\ -1 & -2 \\ 3 & 6 \end{bmatrix}$$

Data Covariance Matrix

$$\Sigma = \frac{1}{3} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} = \frac{1}{3} \begin{bmatrix} -2 & -1 & 3 \\ -4 & -2 & 6 \end{bmatrix} \begin{bmatrix} -2 & -4 \\ -1 & -2 \\ 3 & 6 \end{bmatrix}$$

$$= \frac{1}{3} \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix}$$

$$= \frac{7}{3} \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix}$$

$$= \begin{bmatrix} \frac{14}{3} & \frac{28}{3} \\ \frac{28}{3} & \frac{56}{3} \end{bmatrix}$$

Eigenvalues

- Solve the characteristic function to get eigenvalues:

$$|\Sigma - \lambda \mathbf{I}| = 0$$

$$\left| \frac{1}{3} \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \right| = 0$$

$$\begin{vmatrix} 14 - 3\lambda & 28 \\ 28 & 56 - 3\lambda \end{vmatrix} = 0$$

$$(14 - 3\lambda)(56 - 3\lambda) - 28^2 = 0$$

$$\lambda_1 = \frac{70}{3}$$

$$\lambda_2 = 0$$

Eigenvectors

- Solve the characteristic function to get eigenvalues:

$$|\Sigma - \lambda \mathbf{I}| = 0$$

$$\left| \frac{1}{3} \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \right| = 0$$

$$\begin{vmatrix} 14 - 3\lambda & 28 \\ 28 & 56 - 3\lambda \end{vmatrix} = 0$$

$$(14 - 3\lambda)(56 - 3\lambda) - 28^2 = 0$$

$$\lambda_1 = \frac{70}{3}$$

$$\lambda_2 = 0$$

- Find the eigenvectors:

$$\Sigma \mathbf{u} = \lambda \mathbf{u}$$

$$(\Sigma - \lambda \mathbf{I}) \mathbf{u} = \mathbf{0}$$

$$\begin{bmatrix} \frac{14}{3} - \lambda & \frac{28}{3} \\ \frac{28}{3} & \frac{56}{3} - \lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} \frac{14}{3} - \frac{70}{3} & \frac{28}{3} \\ \frac{28}{3} & \frac{56}{3} - \frac{70}{3} \end{bmatrix} \begin{bmatrix} u_1^{(1)} \\ u_2^{(1)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} \frac{14}{3} - 0 & \frac{28}{3} \\ \frac{28}{3} & \frac{56}{3} - 0 \end{bmatrix} \begin{bmatrix} u_1^{(2)} \\ u_2^{(2)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

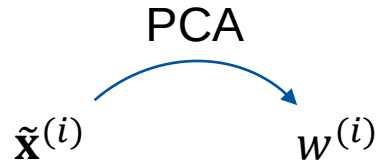
$$\begin{bmatrix} u_1^{(k)} \\ \vdots \\ u_1^{(k)} \end{bmatrix} = \mathbf{u}_k$$

$$\mathbf{u}_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\mathbf{u}_2 = \frac{1}{\sqrt{5}} \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

Data Projection

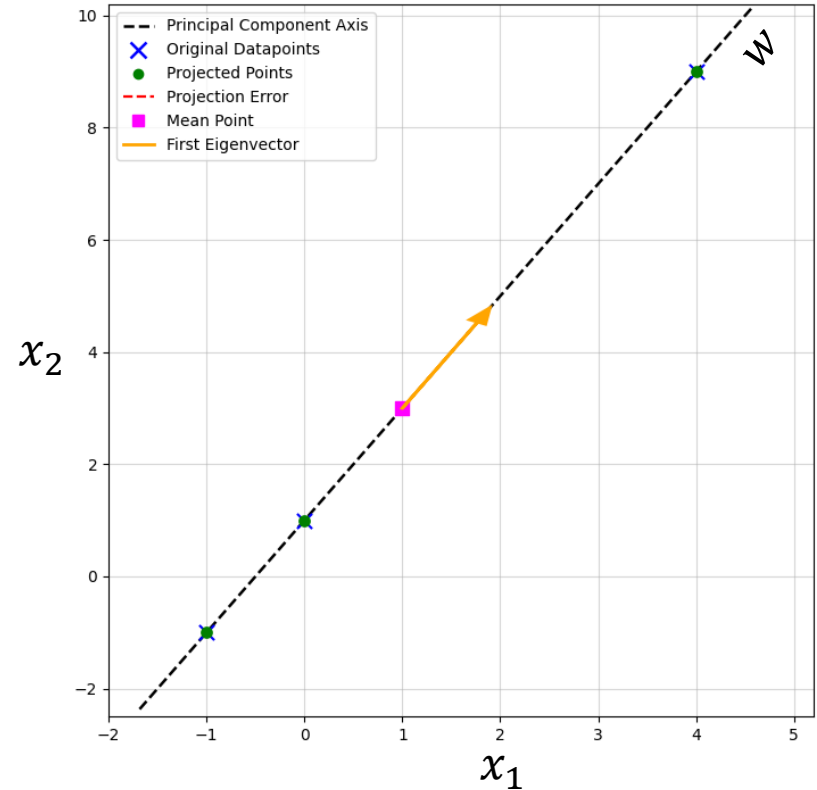
- We now map our data from original 2D space to the 1D space spanned by the first eigenvector



$$w^{(1)} = \tilde{\mathbf{x}}^{(1)} \cdot \mathbf{u}_1 = \begin{bmatrix} -2 \\ -4 \end{bmatrix} \cdot \begin{bmatrix} \frac{1}{\sqrt{5}} \\ 2 \\ \frac{1}{\sqrt{5}} \end{bmatrix} = -\frac{10}{\sqrt{5}} = -4.47$$
$$w^{(2)} = \tilde{\mathbf{x}}^{(2)} \cdot \mathbf{u}_1 = \begin{bmatrix} -1 \\ -2 \end{bmatrix} \cdot \begin{bmatrix} \frac{1}{\sqrt{5}} \\ 2 \\ \frac{1}{\sqrt{5}} \end{bmatrix} = -\frac{5}{\sqrt{5}} = -2.24$$
$$w^{(3)} = \tilde{\mathbf{x}}^{(3)} \cdot \mathbf{u}_1 = \begin{bmatrix} 3 \\ 6 \end{bmatrix} \cdot \begin{bmatrix} \frac{1}{\sqrt{5}} \\ 2 \\ \frac{1}{\sqrt{5}} \end{bmatrix} = \frac{15}{\sqrt{5}} = 6.71$$

Illustration

- Note that, for illustration purpose, we intentionally selected a trivial example in which all datapoints lie on a single line



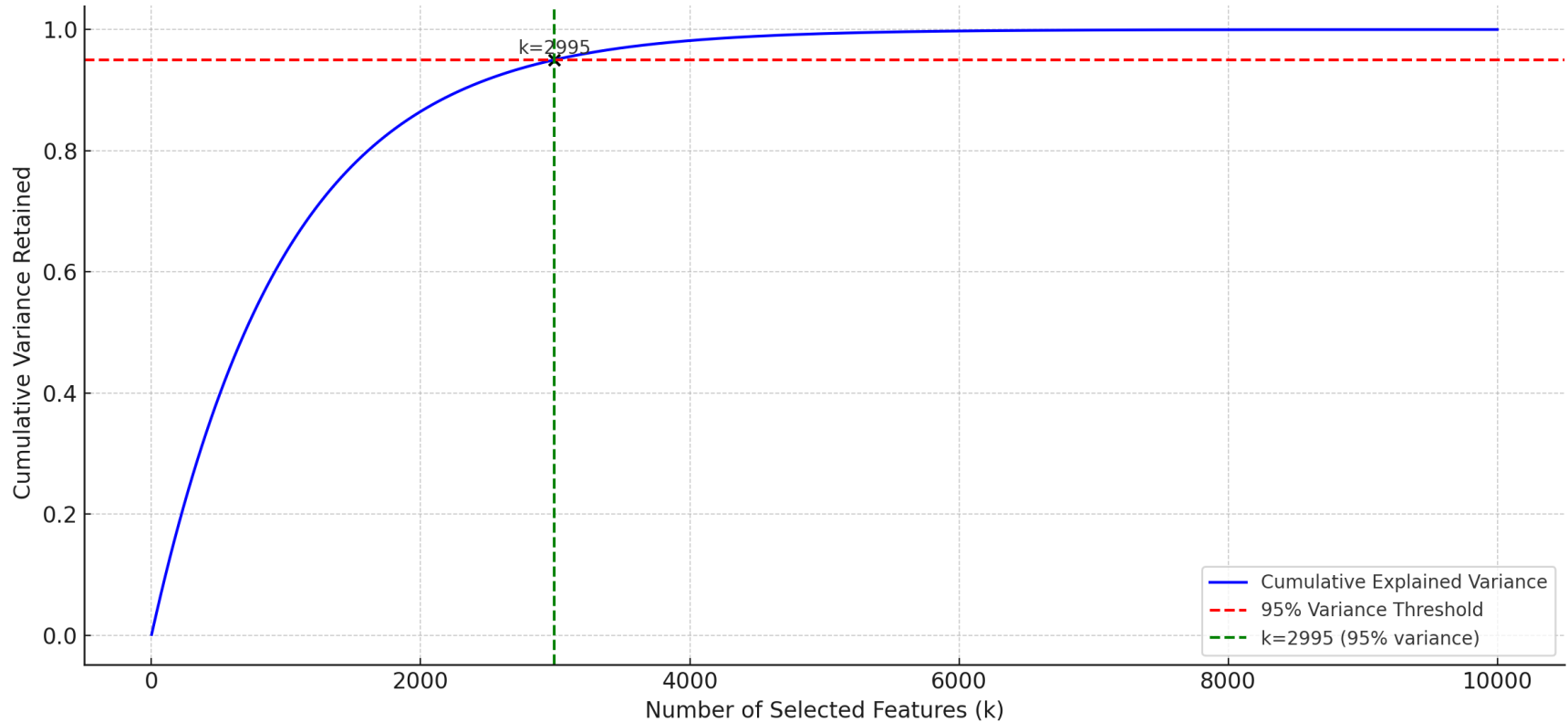


How to choose the number of features?

Variance Retain by PCA

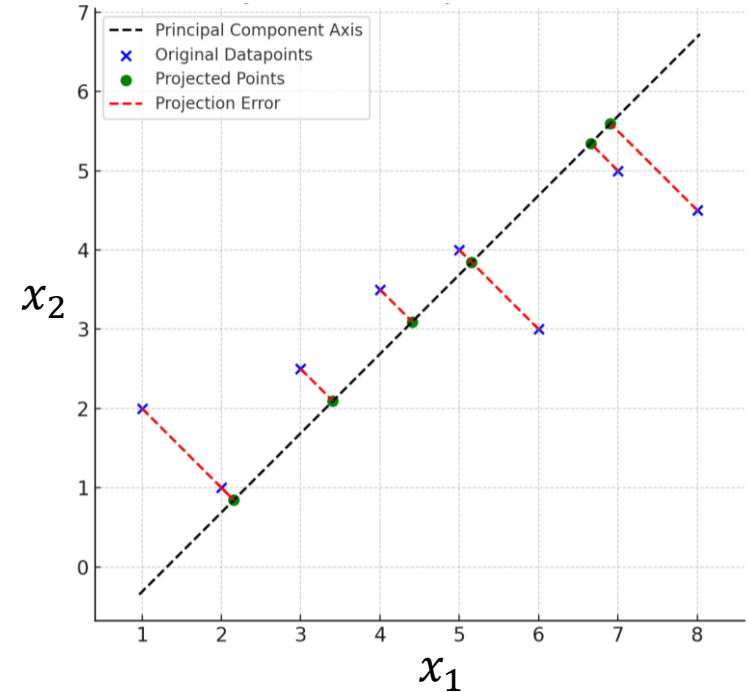
- A natural question that can arise is which value should we chose for the number of features selected by the PCA algorithm (i.e., what is the value of k ?)
- For this, we can do experiment, by varying k between 1 and n and plot the retain variance in the data after applying PCA
- Then, we chose the value of k that retains for example 95% variance of the data

Variance Retain by PCA



Minimum Error Formulation for PCA

- Another alternative way to look at PCA is to minimize the **error between the original data** (blue crosses) **and the projected data** (the green dots)
- That is, we minimize the mean square of the error bars illustrated by the red dashed line segments in the figure

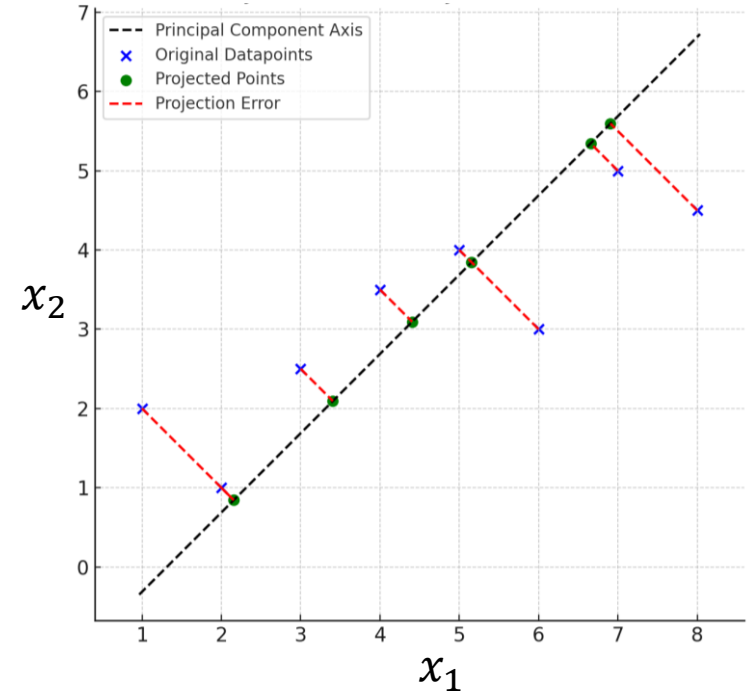


Minimum Error Formulation for PCA

$$\min_{\mathbf{u}} \frac{1}{m} \sum_{i=1}^m \|\mathbf{x}^{(i)} - \mathbf{x}^{(i)} \cdot \mathbf{u}\|^2$$

Subject to: $\|\mathbf{u}\| = 1$

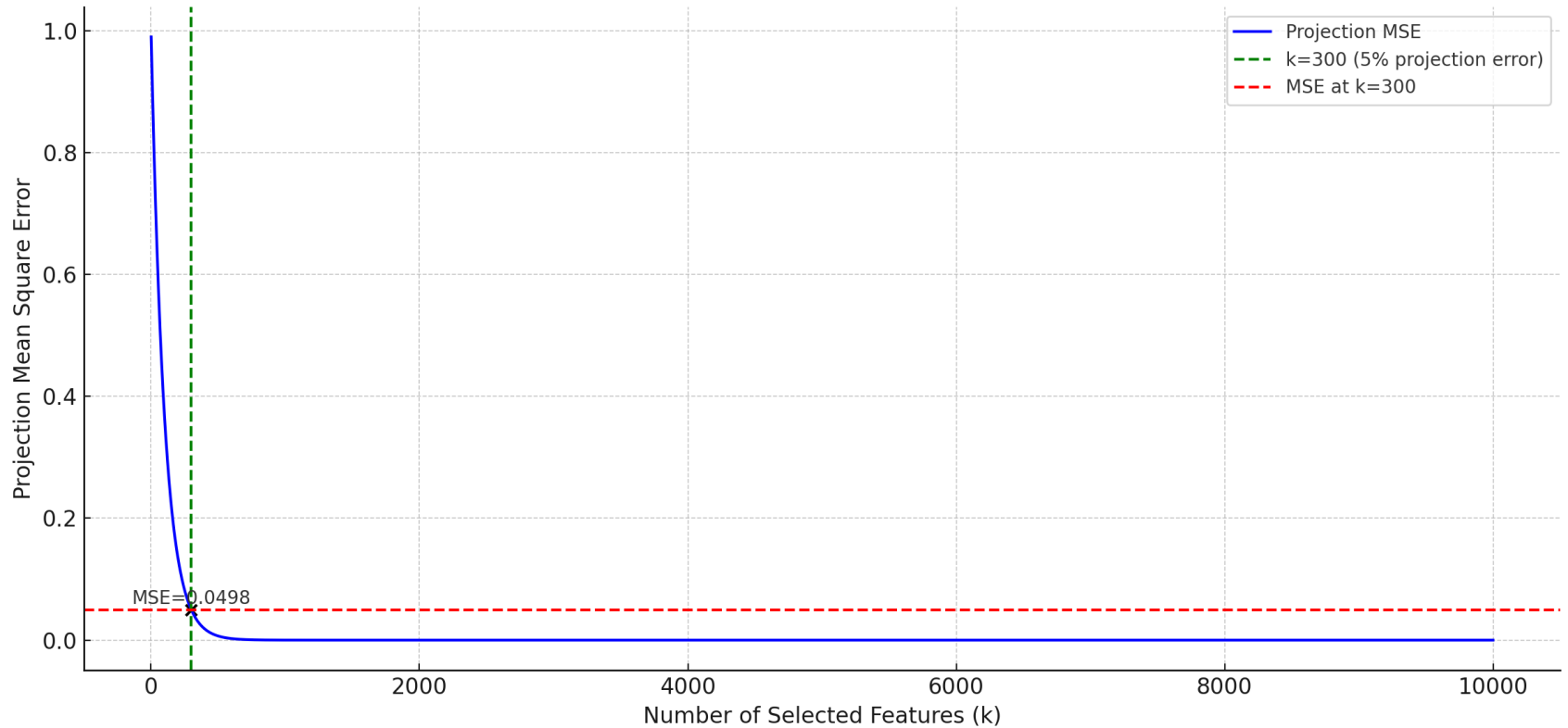
- It can be proved that this optimization problem also ends up with the same solution as the previous representation
- Try this out yourself at home



Minimum Error Formulation for PCA

- Alternative to variance retain in the constructed data by PCA, we now look at the error between the projected data and selected data for different values of k , and select a value that results in say 5% projection error
- That is, after projecting our data from the original space to the subspace spanned by the top k eigenvectors, the different between the original and new datasets is just 5%
- Similarly, to pick another proper value for k , we can plot the projection error when we vary k between 1 and n

Projection Mean Square Error of PCA

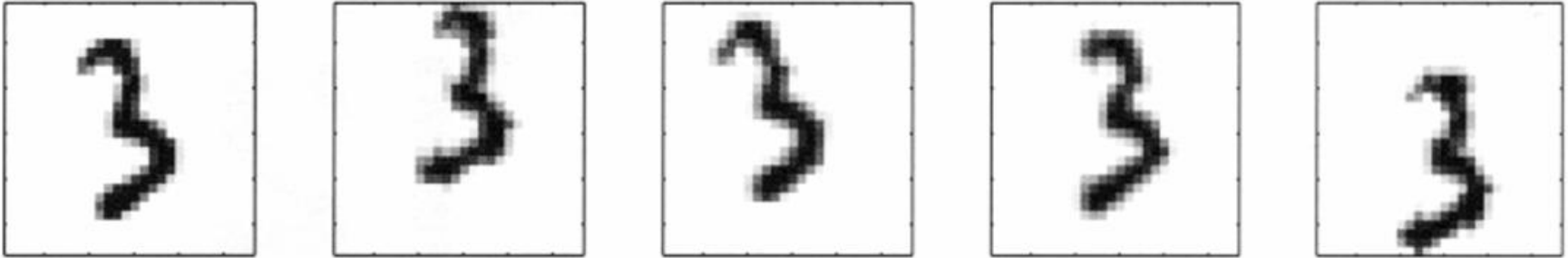




PCA application example

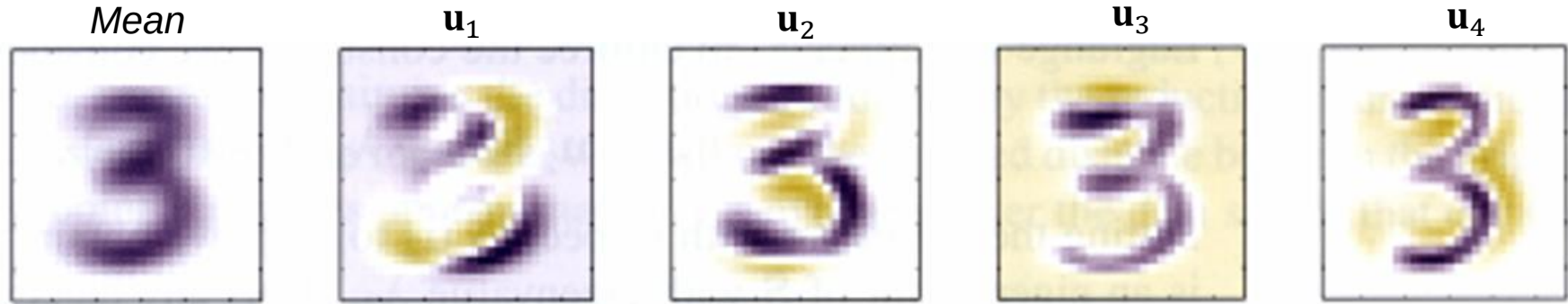
Feature Reduction for Handwritten Digit Detection

- Consider an **artificial dataset** constructed by taking a handwritten digit (say digit 3), represented by a 64 x 64 pixel grey-level image, and embedding it in a larger image of size 100 x 100
- For this, we pad the image with pixels having the value zero (corresponding to white pixels) in which the location and orientation of the digit is varied at random
- Here are some examples from this **dataset with $100 \times 100 = 10k$ features**



Top k Eigenvectors

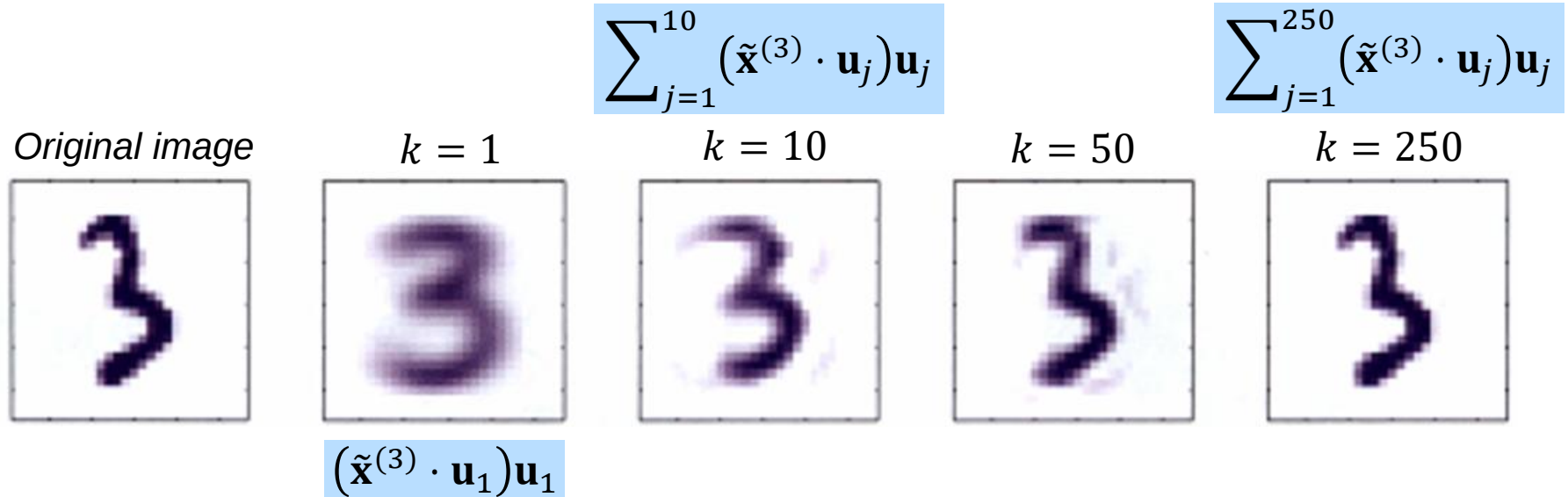
- Here are the mean and the **top four eigenvectors** of this dataset



- This shows that **four features** constructed by PCA already contain a lot of information from the original 10,000 features
- Therefore, using PCA, we can save a lot in the **storage space** and in the **complexity** of the target ML models

PCA Output

- Now, looking back into the artificial dataset that we considered for the digit 3, we can draw the image constructed by different values of k (the number of selected features by PCA)

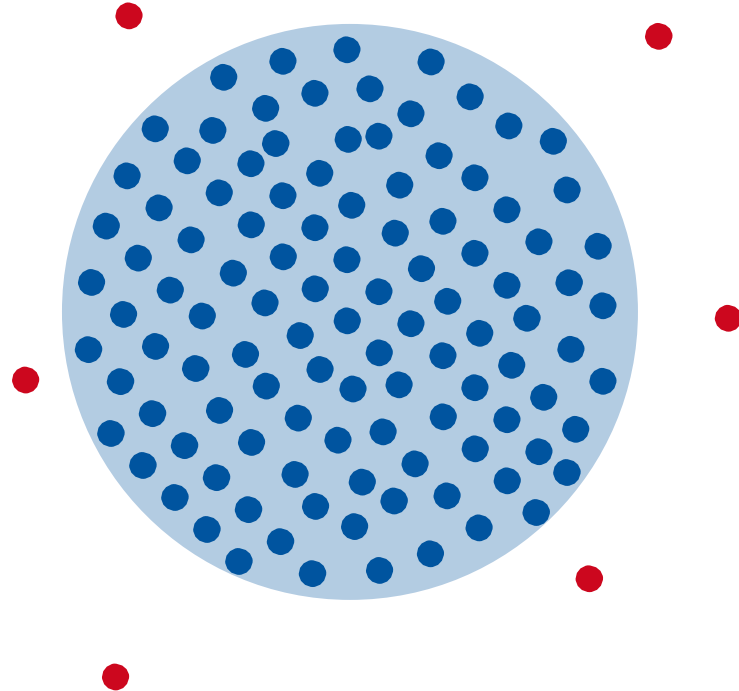




Anomaly detection algorithms

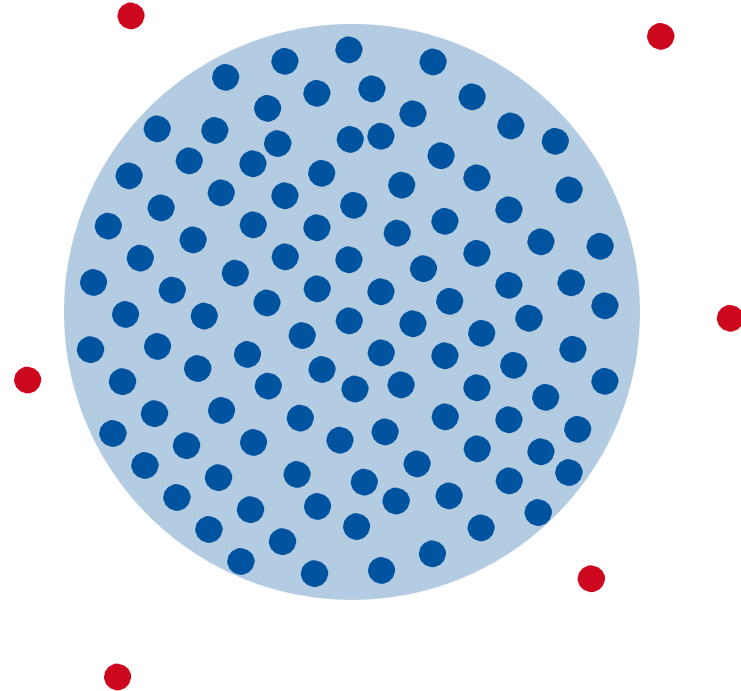
Anomaly Detection

- The objective is to detect anomalies (outliers)
- The data is often benign and might be accompanied with few malicious data
- We may use semi-supervised algorithms to train a model that fits the benign data



Anomaly Detection

- We calibrate the model with few malicious data
- If a test example doesn't fit the trained model, it flags anomaly
- **Applications:** quality inspection, fraud detection, network security



Gaussian Model for Anomaly Detection

To simplify, assume that the features x_j ($j = 1, \dots, n$) are iid random variables with distribution $\mathcal{N}(\mu_j, \sigma_j^2)$

Step 1: Choose features x_j that might be indicative of anomalous examples

Step 2: Use sample mean to estimate the mean and variance for every feature:

Gaussian Model for Anomaly Detection

To simplify, assume that the features x_j ($j = 1, \dots, n$) are iid random variables with distribution $\mathcal{N}(\mu_j, \sigma_j^2)$

Step 1: Choose features x_j that might be indicative of anomalous examples

Step 2: Use sample mean to estimate the mean and variance for every feature:

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$

Gaussian Model for Anomaly Detection

To simplify, assume that the features x_j ($j = 1, \dots, n$) are iid random variables with distribution $\mathcal{N}(\mu_j, \sigma_j^2)$

Step 1: Choose features x_j that might be indicative of anomalous examples

Step 2: Use sample mean to estimate the mean and variance for every feature:

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$

$$\sigma_j^2 = \frac{1}{m} \sum_{i=1}^m \left(x_j^{(i)} - \mu_j \right)^2$$

Gaussian Model for Anomaly Detection

Step 3: Given a new example $\mathbf{x}^{(i)}$, compute $p(\mathbf{x}^{(i)})$:

$$p(\mathbf{x}^{(i)}) = p(x_1^{(i)}, \dots, x_n^{(i)}) = \prod_{j=1}^n p(x_j^{(i)}; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi}\sigma_j} e^{-\frac{(x_j^{(i)} - \mu_j)^2}{2\sigma_j^2}}$$

Gaussian Model for Anomaly Detection

Step 3: Given a new example $\mathbf{x}^{(i)}$, compute $p(\mathbf{x}^{(i)})$:

$$p(\mathbf{x}^{(i)}) = p(x_1^{(i)}, \dots, x_n^{(i)}) = \prod_{j=1}^n p(x_j^{(i)}; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi}\sigma_j} e^{-\frac{(x_j^{(i)} - \mu_j)^2}{2\sigma_j^2}}$$

Step 4: If $p(\mathbf{x}^{(i)}) < \epsilon$, flag anomaly (e.g., raise the alarm)

Multivariate Gaussian Model for Anomaly Detection

- If the feature are dependent, we can fit a multivariate Gaussian model

$$\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)}$$
$$\boldsymbol{\Sigma} = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}^{(i)} - \boldsymbol{\mu})^T (\mathbf{x}^{(i)} - \boldsymbol{\mu})$$

Multivariate Gaussian Model for Anomaly Detection

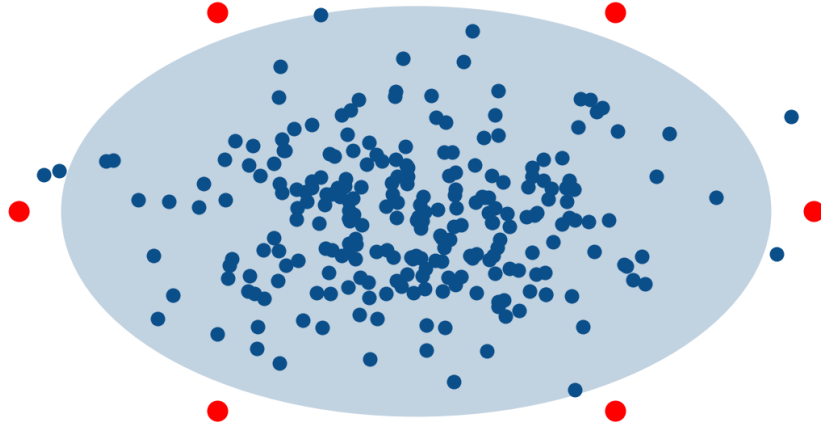
- For a new example $\mathbf{x}^{(i)}$, we compute $p(\mathbf{x}^{(i)})$ as follows

$$p(\mathbf{x}^{(i)}) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x}^{(i)} - \boldsymbol{\mu}) \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu})^T \right)$$

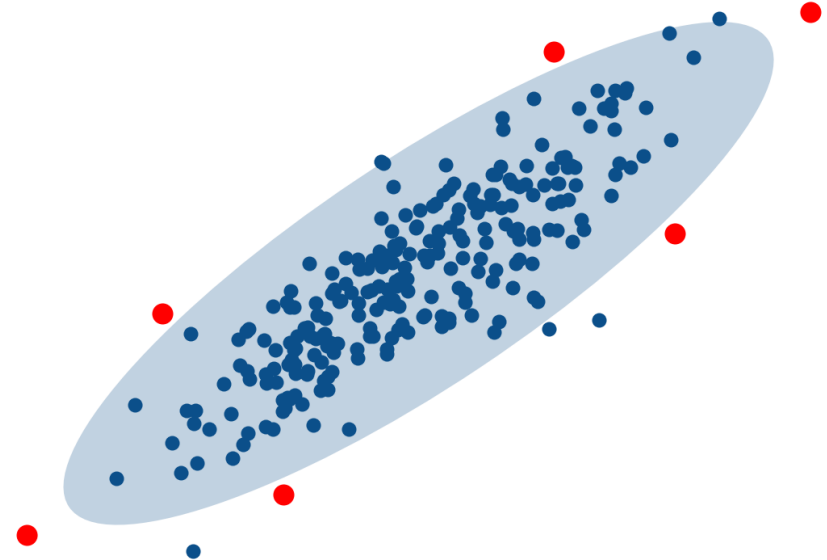
- If $p(\mathbf{x}^{(i)}) < \epsilon$, we flag anomaly

Multivariate Gaussian Model for Anomaly Detection

Diagonal Covariance Matrix

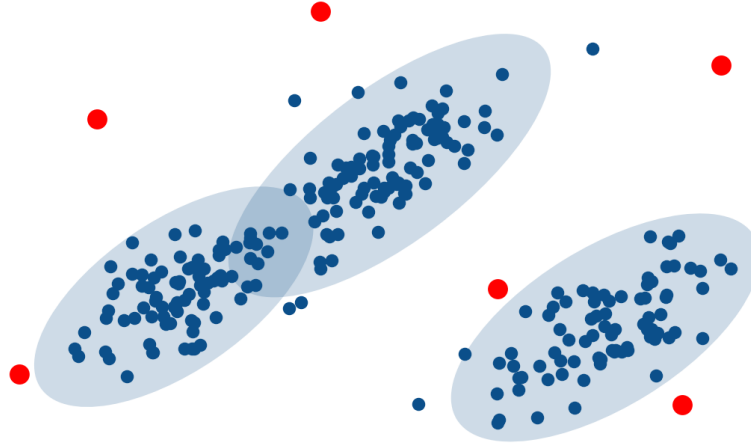


Full Covariance Matrix



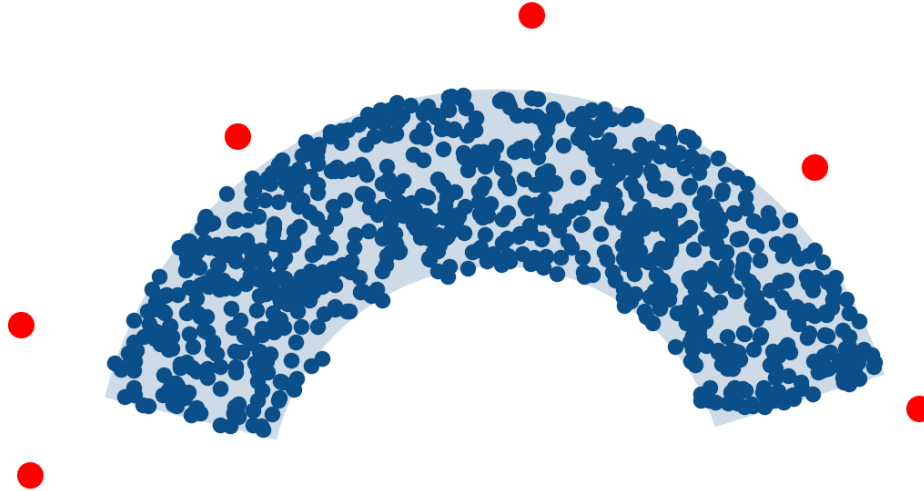
Gaussian Mixture Model for Anomaly Detection

- We may also try to use other unsupervised learning models for anomaly detection, e.g., Gaussian Mixture Model (GMM)



DBSCAN Algorithm for Anomaly Detection

- We may also use DBSCAN algorithm to construct an anomaly detection model
- It is helpful when we have irregular (cluster) shape(s) for benign data





Metrics to assess anomaly detection algorithms

Performance Metrics for Anomaly Detection Algorithm

	Predicted Anomaly	Predicted Benign
Actual Anomaly	True Positive (TP)	False Negative (FN)
Actual Benign	False Positive (FP)	True Negative (TN)

← *Confusion matrix*

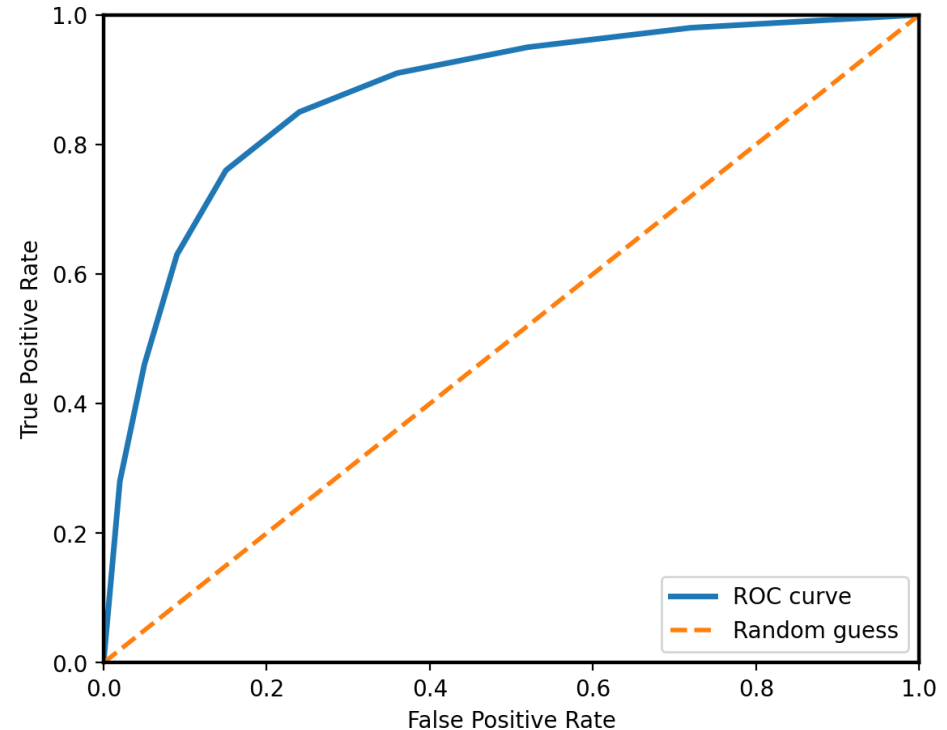
- **True Positive Rate (TPR) / Recall / Sensitivity** = $\frac{TP}{TP+FN}$ (of all actual anomalies, how many did we correctly detect?) → Did we catch all anomalies?
- **Precision** = $\frac{TP}{TP+FP}$ (of all raised alarms, how many are correct?) → Are our alarms trustworthy?

Performance Metrics for Anomaly Detection Algorithm

- **False Positive Rate (FPR)** = $\frac{FP}{FP+TN}$ (fraction of benign examples incorrectly flagged as anomalies)
- **Specificity** = $\frac{TN}{TN+FP} = 1 - FPR$ (fraction of all benign examples (negatives) correctly rejected by the model)
- **Accuracy** = $\frac{TP+TN}{TP+TN+FP+FN}$ (overall correctness)

Receiver Operating Characteristics (ROC)

- ROC plots TPR vs FPR
- It shows performance of a detector across all decision thresholds
- ROC curve shows the trade-off between detecting positives and avoiding false alarms across all thresholds
- The steeper the curve, the better the performance of the detector



Performance Metrics for Anomaly Detection Algorithm

- Note that in anomaly detection application, the majority of data is benign, e.g., 99% benign + 1% anomaly (class imbalance)
- In such setting, accuracy is often a misleading metric
- Furthermore, a model that predicts everything as normal has very high accuracy (99%), but it is completely useless
- Therefore, we have to be careful which metric we are using to assess our model

Performance Metrics for Anomaly Detection Algorithm

- What to focus?
 - **Recall (sensitivity or TPR)** = $\frac{TP}{TP+FN}$ (the higher, the less we miss anomalies)
 - **Precision** = $\frac{TP}{TP+FP}$ (the higher, the more trustworthy alarms)
- Performance Trade-offs:
 - if missing anomalies is critical → prioritize recall
 - If false alarms are costly → prioritize precision

Next Lecture (04.05.2026)

- Supervised Learning

Thank you!

